Quantifying Sheboygan's "Lake Effect"

Jackson Pond

https://qr.codes/FzDOVl

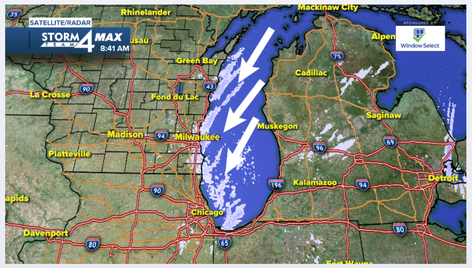
GITHUB REPO

Theory of Predictive Modeling

CS 580/PHSCS 513R

The Lake Effect

There is perhaps no greater fodder for small talk in Sheboygan than the storied "Lake Effect." All weather changes, both gradual and sudden, are attributed at some point to the lake effect, which is the blanket name for the kinds of weather phenomena directly related to Sheboygan's proximity to the great lake Michigan. It has such a powerful hold on the public's imagination that a radio station, a line of merch, and a cafe are all named after it.

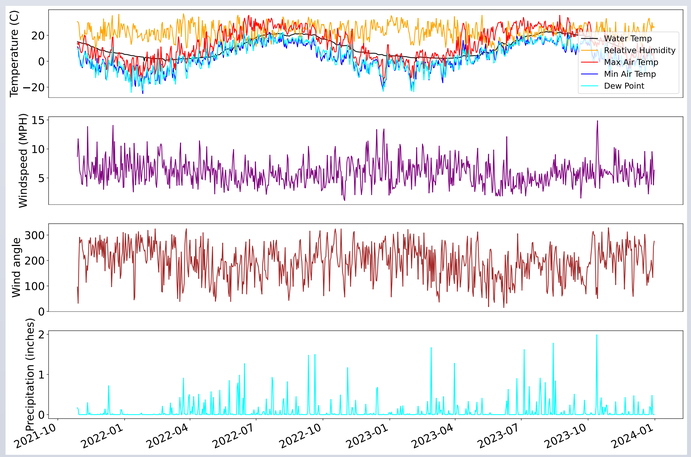


It is well documented that cities along Michigan's western coastline receive more precipitation. The question this poster attempts to answer is how the lake factors in when predicting whether precipitation will fall on a given day in Sheboygan, Wisconsin.

Eyeballing the Data

The data used for this project was procured from three sources: An airport weather station whose data is hosted at Iowa State[1], an NDBC buoy near Sheboygan[2], and a GLSEA databank[3]. All atmospheric and lake-observation datasets (ASOS, NDBC SGNW3, and GLSEA) originate from NOAA observational or research programs. Missing and/or faulty data was an issue for the buoy and GLSEA data. The weather station consisted of daily values (averages, maximum, and minimum) while the other sources consisted of hourly values.

For the buoy data, for gaps of less than a day, I linearly interpolated between known points. I simply dropped gaps longer than a day. The largest swath of data dropped for this purpose was missing pressure values. For each feature, I clipped data to reasonable ranges to correct the most egregious faulty readings. Since none of the sensors had integrity indicators, I did not attempt to correct faulty readings further. Hourly readings were averaged over days in order to match weather station data.



The features I used for fitting a predictive model are **Day of the Year** to capture seasonality, **whether it rained/snowed the previous day**, **Avg. Relative Humidity**, **Maximum Dew point, Minimum Dew point** (from weather station)**, Avg. Air Pressure, Avg. Windspeed, Avg. Wind Direction** (from buoy 1)**,** and **Average Water Temperature** near shore (from buoy 2).

Unfortunately, the only water temperature data available only covers October 2021 - December 2024. This was much more limited than the rest of the features, and as such, when factoring in lake effect the model is fit on a much smaller dataset.

SOURCES

[1] Iowa Environmental Mesonet. (2000–2024). *SBM ASOS daily climate data (WI_ASOS)*. Iowa State University. Retrieved from https://mesonet.agron.iastate.edu/

[2] National Data Buoy Center. (2000–2024). *SGNW3 – Sheboygan, WI nearshore environmental data*. NOAA. Retrieved from https://www.ndbc.noaa.gov/.

[3] NOAA Great Lakes Environmental Research Laboratory. (2000–2024). *Great Lakes Surface Environmental Analysis (GLSEA) water temperature*. NOAA GLERL. Retrieved from https://www.glerl.noaa.gov/.

Logistic Regression

Predicting whether it will rain or snow is a binary classification problem. As such, we wish to train a regression model that takes its values in the interval [0, 1], such as a *logistic regression* model. Logistic regression relies on the logistic function. Let Y be a random variable indicating whether it precipitates on Sheboygan on a given day. Suppose that

$$Y \mid X \sim \text{Bernoulli}(\text{sigm}(\mathbf{X}^T\beta))$$

So that

$$P(Y \mid \mathbf{X}) = \frac{1}{1 - \exp(-\mathbf{X}^T\beta)}$$

Then the maximum likelihood estimate is found by minimizing the objective function:

$$\hat{\beta} = \underset{\beta}{\text{argmin}}\ \ell(\beta \mid D) = \underset{\beta}{\text{argmin}} \sum_{i=1}^{n} \log(1 + e^{z_i}) - y_i z_i$$

I walk through this derivation in more detail in the src/write-up/log-reg.md file in the GitHub repository.

I. Logistic Regression Without Lake Effect

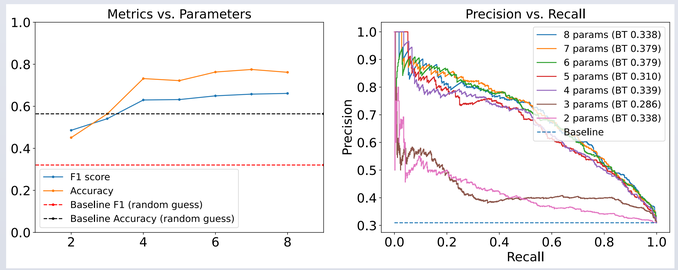
The original bucket of models I searched through had the following form:

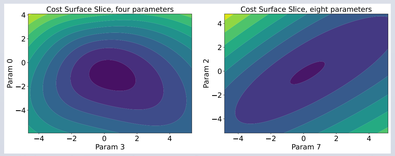
$$P(Y = 1) = \text{logit}^{-1}(\beta_0 + \beta_1 \sin(\frac{2\pi d}{365}) + \beta_2 \cos(\frac{2\pi d}{365}) + \beta_3 r_{t-1} + \beta_4 h + \beta_5 p + \beta_6 v + \beta_7 \rho)$$

Where $\mathbf{d}$ is day of the year, $\mathbf{r_{t-1}}$ is a binary indicator of whether it rained the day before, $\boldsymbol{h}$ is the relative humidity, $\boldsymbol{p}$ is the pressure, $\boldsymbol{v}$ is windspeed, and ρ is the dew point. I implemented a custom class that to find the optimal model minimizing the above objective function, and to find the hessian of the cost surface at the minimizer.

Sloppy Model Analysis

Inspection of the hessian of the cost surface of at the minimizer revealed which directions were sloppy and which were steep. I removed the sloppiest parameter, retrained, and repeated.





Following this method I reduced the model from eight parameters to four without loss of predictive power. The four steep parameters were relative humidity, maximum dew point, windspeed, and an intercept. At four parameters the hessian is well conditioned. Dropping any feature causes the model to greatly drop in accuracy, below the accuracy of always guessing no precipitation.

I also performed straightforward PCA on the three dimensional data, ignoring the intercept (relative humidity, windspeed, maximum dew point) and produced the figure to the right. The data groups by category but is not near separable. As expected for logistic regression, the points firmly in one region or the other are predicted correctly consistently. However the instances in the middle muddle are less consistently predicted correctly.

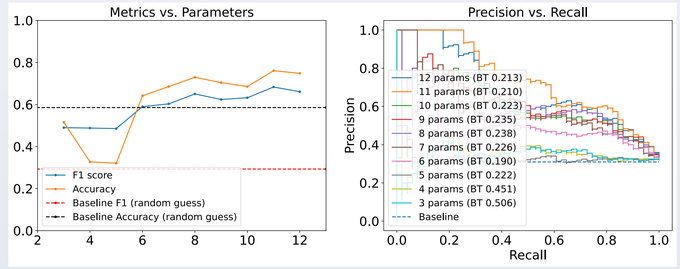
As can be seen in the data exploration figure on the left panel, many of the features involved in the eight parameter model are highly correlated, and it makes sense that highly dependent—and therefore sloppy—directions were dropped from the model.

II. Logistic Regression With Lake Effect

I also found a logistic regression fit when including lake effect factors in the model, namely **water temperature**, **wind direction**, and a **blowing-on-shore binary indicator**. These are represented by ***wt***, θ, and $\mathbf{I_O}$ respectively.

$$P(Y = 1) = \text{logit}^{-1}(\beta_0 + \beta_1 \sin(\frac{2\pi d}{365}) + \beta_2 \cos(\frac{2\pi d}{365}) + \beta_3 r_{t-1} + \beta_4 h + \beta_5 p + \beta_6 v + \beta_7 \rho + \beta_8 \sin(\theta) + \beta_9 \cos(\theta) + \beta_{10} wt + \beta_{11} I_O)$$

I fit these models similarly, removing one sloppy direction at a time. This didn't go as well for the lake effect model as for the normal model. The hessian of the cost surface at the minimizer for the six parameter model is still poorly conditioned, but removing the final sloppy direction causes the drop in accuracy seen in the figure below. Additionally, the loss in predictive power between the 12 parameter model and the six parameter model is greater than in the normal model.

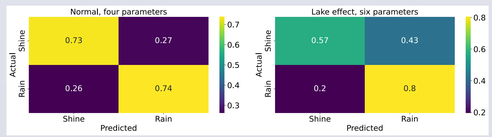


The six parameters in the six parameter lake effect model were 1, ρ, $\sin(\theta)$, $\cos(\theta)$, wt, and I_O, which are all of the lake effect parameters with an intercept feature and dew point. Dew point is a sloppy direction. I was pleasantly surprised to see that the lake effect parameters were the stiffest. That may indicate they are the most useful to predict precipitation, or it could suggest they are highly correlated with other useful variables and both are not needed.

Conclusion

I. Results

The four parameter normal model performed better than the six parameter lake effect model, with an respective accuracies of 74.84% and 64.15% and respective F1 scores of 0.7260 and 0.5899. However, the lake effect model was trained on a dataset less than a tenth the size of the normal model.



When training on a dataset the same size as the lake effect model, the normal model achieved similar accuracy of about 74%.

The sloppy model analysis revealed that many of the periodic features were redundant and dependent on one another, and only one was needed. Interestingly, the steep variables in the normal model were not the steep variables in the lake effect model. The normal model was reduced from 11 parameters to four via sloppy analysis, and the lake effect model from 12 to six. A major issue with the method I used is that there is no way to separate the lake effect from any variables I used, even in the normal "non-lake" model.

II. Future Work

This project suffered from a lack of complete historical data. Improved future efforts will depend on the accuracy and availability of more complete data, such as for water temperature. Most features I wanted in my model were available, but a more complete model would also include temperatures at several altitudes to capture effects from convection, as well as fetch length of wind over the lake.

Nonetheless this is a fun dataset that is well equipped for lots of different kinds of models. As part of my exploration, I created a simple binary Markov chain model. More complicated kinds of Markov chains could be explored. I would also like to train ARIMA/SARIMA models, which are typical time-series models. In a different direction, regression on the *amount* of precipitation when it does fall could be a fruitful direction.

Gratifyingly a naively trained RandomForestClassifier from scikit-learn underperformed both logistic regression models, with accuracy 63% and F1 score of 0.57.